

Retrieval-Augmented Generation: A Practical Overview

Why Retrieval-Augmented Generation Exists

Large language models are trained on a fixed snapshot of data and have no inherent way to access information created after training, private documents, or highly specific facts buried in a large corpus. Retrieval-Augmented Generation, or RAG, addresses this by pairing the language model with a retrieval step: relevant passages are pulled from an external knowledge source at query time and inserted into the model's context window before it generates an answer. This lets a general-purpose model answer questions about, for example, a company's internal documentation or a user's uploaded PDFs, without retraining the model itself.

Chunking Strategy

Before documents can be retrieved, they must be broken into chunks small enough to embed meaningfully and to fit within a prompt alongside a user's question. Chunk size involves a tradeoff: chunks that are too large dilute the specific information a query is looking for, while chunks that are too small lose surrounding context needed to interpret them correctly. Overlapping chunks, where each chunk shares a small amount of text with its neighbor, help prevent important sentences from being split awkwardly across a chunk boundary.

Embeddings and Vector Similarity

Once text is chunked, each chunk is converted into a numerical vector, called an embedding, using an embedding model. These vectors are designed so that chunks with similar meaning end up close together in vector space, even if they use different wording. When a user asks a question, the question itself is embedded the same way, and a similarity search, typically cosine similarity or dot product, finds the stored chunks whose vectors are closest to the query vector. These are the chunks considered most relevant to the question.

Vector Databases

Storing and searching embeddings efficiently at scale requires a vector database, such as ChromaDB, Pinecone, or FAISS. These systems index vectors so that similarity search remains fast even across millions of chunks, using approximate nearest neighbor algorithms rather than brute-force comparison. For a small personal project, a local vector database like ChromaDB is sufficient and avoids any cloud hosting cost, while production systems handling large document sets often move to a managed, distributed vector database for scalability.

Generating the Final Answer

After the most relevant chunks are retrieved, they are combined with the original question into a single prompt sent to the language model, often with an instruction to answer only using the provided context. This grounds the model's response in real source material and makes it possible to cite exactly which

chunk, and therefore which document and page, supported a given answer. This citation capability is one of the most valuable features of a well-built RAG system, since it lets a user verify the answer rather than trusting the model blindly.